



COURS DJANGO — IFNTI SOKODÉ

Modéliser le domaine

02

POUR COMMENCER

Quiz sur la séance Mise en route — 15 minutes, sans machine.

Au programme

01 Du domaine au schéma

entités, relations, et ce que chaque choix coûte

03 L'héritage

abstrait ou concret : une table, ou pas

05 Contraintes et lisibilité

choix, unicité, Meta, __str__

07 Le shell

peupler et interroger la base en Python

02 Un modèle par fichier

models/ en package

04 Les relations

ForeignKey, ManyToMany, OneToOne, et on_delete

06 Lire le SQL

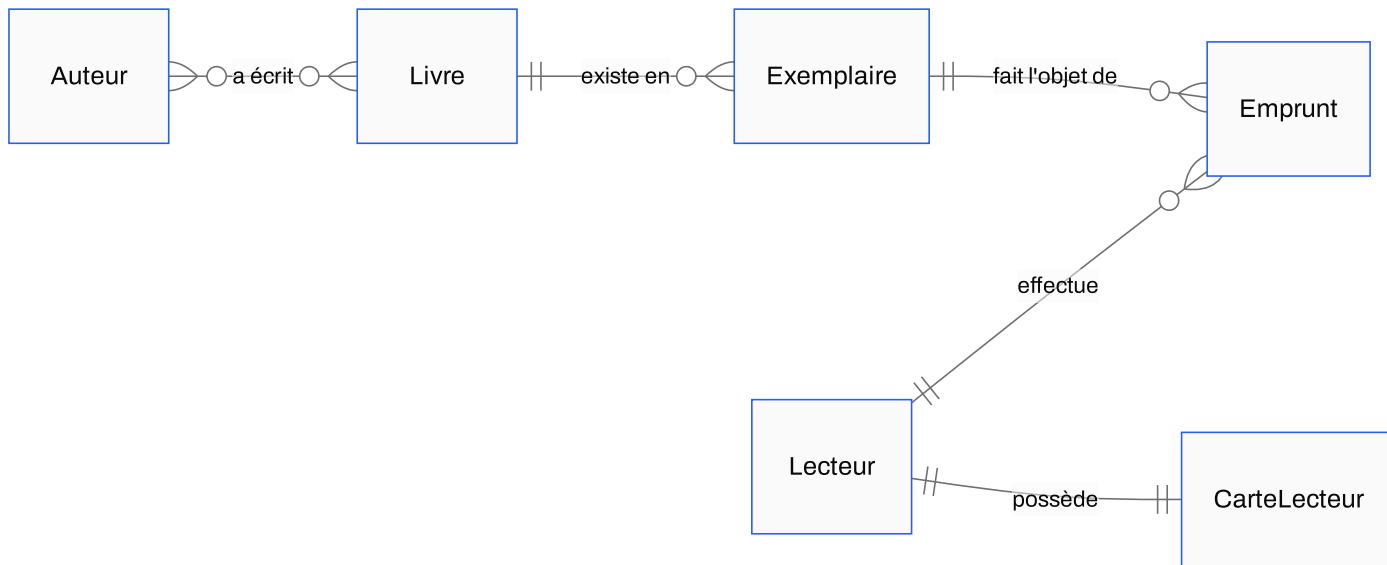
ce que Django crée vraiment



01

Du domaine au schéma

La bibliothèque de l'IFNTI



Pas les stages — ceux-là, c'est vous qui les modéliserez au TP.

À RETENIR

Chaque trait du schéma est une décision. Chaque décision a un coût, aujourd'hui ou dans trois mois.



02

Un modèle par fichier

models.py devient un package

```
└─ bibliotheque
   └─ models
      ├── __init__.py
      ├── auteur.py
      ├── livre.py
      ├── exemplaire.py
      ├── lecteur.py
      ├── carte_lecteur.py
      └── emprunt.py
   └─ admin.py
```

Un fichier par modèle. On retrouve un modèle par son nom, et deux personnes qui travaillent sur deux modèles ne se marchent plus dessus dans git.

`__init__.py` **réexporte tout.** Le reste du code continue d'écrire `from .models import Livre`, sans savoir que le fichier a éclaté.

Un `__init__.py` qui réexporte

```
# bibliotheque/models/__init__.py
from .auteur import Auteur
from .carte_lecteur import CarteLecteur
from .emprunt import Emprunt
from .exemplaire import Exemplaire
from .lecteur import Lecteur
from .livre import Livre

__all__ = ["Auteur", "CarteLecteur", "Emprunt", "Exemplaire", "Lecteur", "Livre"]
```



03

L'héritage

Abstrait ou concret ?

abstract = True

- Aucune table pour le parent
- Les champs sont recopiés dans chaque enfant
- Pas de requête sur « tous les documents »
- Lectures simples, sans jointure

OU

Héritage multi-table

- Une table pour le parent, une par enfant
- Chaque enfant pointe vers son parent
- On peut interroger tous les documents d'un coup
- Une jointure à chaque lecture d'un enfant

Un parent abstrait

```
class Document(models.Model):
    titre = models.CharField(max_length=200)
    annee = models.PositiveSmallIntegerField()

    class Meta:
        abstract = True
        ordering = ["titre"]

class Livre(Document):
    isbn = models.CharField(max_length=13, unique=True)

    class Meta(Document.Meta):
        verbose_name = "livre"
```

- 1 **abstract = True** — Document n'aura pas de table. Il ne sert qu'à factoriser des champs.
- 2 **Meta hérite aussi** — si l'enfant redéfinit Meta, il doit hériter de celle du parent, sinon il perd l'ordre par défaut.
- 3 **Livre** — sa table contiendra titre, annee et isbn, comme s'ils avaient été écrits là.



04

Les relations

Dire combien, de chaque côté

ForeignKey

- plusieurs vers un
- un exemplaire → un livre
- un livre → plusieurs exemplaires
- la clé est du côté « plusieurs »

ManyToManyField

- plusieurs vers plusieurs
- un livre → plusieurs auteurs
- un auteur → plusieurs livres
- une table de jonction, créée pour vous

OneToOneField

- un vers un
- un lecteur → une carte
- une carte → un lecteur
- une clé étrangère unique

related_name, ou le chemin du retour

```
class Exemplaire(models.Model):
    livre = models.ForeignKey(
        Livre, on_delete=models.PROTECT,
        related_name="exemplaires")

class Livre(Document):
    auteurs = models.ManyToManyField(
        Auteur, related_name="livres")

class CarteLecteur(models.Model):
    lecteur = models.OneToOneField(
        Lecteur, on_delete=models.CASCADE,
        related_name="carte")
```

- 1 **ForeignKey** — `exemplaire.livre` donne le livre. Dans l'autre sens, `livre.exemplaires.all()` donne ses exemplaires, grâce à `related_name`.
- 2 **ManyToManyField** — `livre.auteurs.all()` et `auteur.livres.all()`. Déclaré une seule fois.
- 3 **OneToOneField** — `lecteur.carte` donne directement la carte, pas une liste.

Que se passe-t-il quand on supprime le parent ?

CASCADE	les enfants sont supprimés avec lui
PROTECT	la suppression est refusée tant qu'il a des enfants
SET_NULL	les enfants perdent leur lien ; le champ doit accepter null
SET_DEFAULT	les enfants pointent vers une valeur par défaut
RESTRICT	comme PROTECT, sauf si le parent part lui-même en cascade

**on_delete
n'est pas un
réglage
technique.
C'est une
*règle métier.***



05

Contraintes et lisibilité

Choix, unicité, tri, affichage

```
class Exemplaire(models.Model):
    class Etat(models.TextChoices):
        BON = "bon", "Bon état"
        ABIME = "abime", "Abîmé"

    livre = models.ForeignKey(Livre, ...)
    numero = models.PositiveSmallIntegerField()
    etat = models.CharField(max_length=10, choices=Etat)

    class Meta:
        ordering = ["livre", "numero"]
        constraints = [models.UniqueConstraint(
            fields=["livre", "numero"],
            name="numero_par_livre")]

    def __str__(self):
        return f"{self.livre} (ex. {self.numero})"
```

- 1 **TextChoices** — une liste fermée de valeurs, avec leur libellé. L'admin affiche une liste déroulante.
- 2 **UniqueConstraint** — le même numéro ne peut exister deux fois pour un même livre. Vérifié par la base elle-même.
- 3 **ordering** — le tri par défaut de toutes les requêtes sur ce modèle.
- 4 **__str__** — ce que l'on voit partout : admin, shell, listes déroulantes.

CE QUE VOUS LIREZ AILLEURS

unique_together

```
class Meta:  
    unique_together = [["livre", "numero"]]
```



L'ancienne écriture, encore acceptée et omniprésente dans les tutoriels. La documentation de Django annonce qu'elle **pourrait être dépréciée** : on écrit `UniqueConstraint`, qui sait aussi nommer la contrainte, la rendre conditionnelle, ou la faire porter sur une expression.



06

Lire le SQL

Une table que vous n'avez pas écrite

```
CREATE TABLE "bibliotheque_livre_auteurs" (  
  "id" integer NOT NULL PRIMARY KEY AUTOINCREMENT,  
  "livre_id" bigint NOT NULL REFERENCES "bibliotheque_livre" ("id"),  
  "auteur_id" bigint NOT NULL REFERENCES "bibliotheque_auteur" ("id")  
);
```



07

Le shell

uv run manage.py shell

● ● ● shell Django

```
$ uv run manage.py shell
12 objects imported automatically
$ hugo = Auteur.objects.create(nom="Hugo", prenom="Victor")
$ livre = Livre.objects.create(titre="Les Misérables", annee=1862, isbn="9782070409228")
$ livre.auteurs.add(hugo)
$ hugo.livres.all()
<QuerySet [<Livre: Les Misérables>]>
```



Depuis Django 5.2, le shell **importe automatiquement vos modèles** : plus besoin d'écrire les `from ... import` à chaque session.

L'ORM en six gestes

`Livre.objects.create(...)` crée et enregistre en une fois

`Livre.objects.get(isbn=...)` un seul objet ; erreur s'il y en a zéro ou plusieurs

`Livre.objects.filter(annee__lt=1900)` une liste filtrée ; `__lt`, `__gte`, `__contains`...

`Exemplaire.objects.filter(livre__auteurs__nom="Hugo")` le double souligné traverse les relations

`livre.auteurs.set([a, b])` remplace les liens d'un ManyToMany

`livre.exemplaires.count()` compte en SQL, sans charger les objets

Ce que vous allez faire pendant deux heures et demie

- Éclater `models.py` en package, et constater ce que Django en pense
- Modéliser **tout le suivi des stages** à partir de la parole de la responsable des stages
- Trancher chaque relation, chaque `on_delete`, chaque contrainte — **et le justifier**
- Lire le SQL de votre migration, et y chercher vos décisions
- Peupler la base par le shell, puis l'interroger



Pas de modèle à recopier aujourd'hui : il n'y en a pas au tableau. Bloqué plus de **dix minutes** ? Appelez.

C'est en forgeant que l'on devient forgeron

Ce cours s'oublie, ce que vous codez reste

L'ÉNONCÉ EST SUR MOODLE – PROCHAINE SÉANCE : VUES, URLS, TEMPLATES